P1292R0
Customization Point Functions

Published Proposal, 2018-10-08

This version:
http://wg21.link/p1292r0

Author:
Matt Calabrese

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes a new language facility for easily specifying, overriding, and invoking single-function customization points that are frequently used in generic libraries, such as `swap` of the standard library, and `AbslHashValue` of Abseil[1]. This proposal is intended to target a standard after C++20.

Table of Contents

1. Introduction

1.1. Example

Before going into details, below is a simple example of the core of what is proposed -- a "virtual", but statically-dispatched, non-member function that declares a logical entry point for user-customization. Users may explicitly override such a function for their type from their own namespace, in which case qualified calls to the customization point will dispatch to the user's customized implementation whenever appropriate.

```
namespace std {

// A customization point named "swap", including a default definition.
template< class T >
virtual void swap( T& lhs, T& rhs ) { /*...*/ }

} // namespace std

// A user's override (may appear in their own namespace)
void swap( foo& lhs, foo& rhs ) override : std::swap { /*...*/ }
```

1.2. Motivation

Many C++ libraries introduce simple, named, single-function customization points through which generic code may interface with user-defined operations. Two common techniques for implementing these customization points are nested members of template specializations (such as with traits classes or with `std::numeric_limits`), and argument dependent lookup (`swap`).

Both of the aforementioned approaches have advantages and disadvantages. The traits-based approach allows for groupings of associated types and associated functions, has a fairly clear declaration, has (relatively) simple rules when matching template arguments to specializations, but is a bit cumbersome to use directly and user-specialization often requires jumping into a foreign namespace.

The ADL-based approach applies to individual functions as opposed to groupings of associated properties, so it is a bit narrower in scope, but in that narrow scope it has some advantages. When a user needs to override such a customization point, they do not need to enter into a foreign namespace nor do they have to specialize a type-template just to customize a single function.

However, the simplicity of ADL-based customization points is in some sense an illusion. Proper creation and use of ADL customization points is so subtle that experts in C++ have written extensive articles about them[2] and they are a frequent source of questions for newcomers and experienced C++ programmers alike[3]. Ranges even goes so far as to introduce the design pattern of a "Customization Point Object" at the library level[4], whose primary purpose is to make it easier to consistently specify and invoke ADL-based customization points. A prominent programmer in the generic programming community has even gone so far as to state ADL is the one feature that he would remove from C++ if he could.[5]

The following are considered by the author of this proposal to be some core issues with customization points based on ADL.

1. They do not have a clear declaration in code.
2. They are subtle to invoke correctly and doing so incorrectly may not always produce a compile-time error (`using std::swap; swap(lhs, rhs);`).
3. Associated-namespace rules are rarely understood, cause confusion, and bugs.
4. Lookup often leads to a large amount of overloads that must be considered.

This proposal attempts to make function-kind customization points a directly supported part of the language, with a simpler set of rules than existing approaches that most programmers would be able to fully understand.

§ 2. Customization Point Functions

Customization point functions are the proposed alternative to the subset of customization points that developers currently use ADL for.

§ 2.1. Declaring a Customization Point Function

A customization point function may be declared at namespace scope just as any other function template, but with a preceding `virtual` keyword. Customization point functions may be constrained just as any other function template may be constrained.

```
template< class T >
virtual constexpr void swap( T& lhs, T& rhs )
  noexcept(    std::is_nothrow_move_constructible_v<T>
            && std::is_nothrow_move_assignable_v<T>
          )
  requires MoveConstructible<T> && MoveAssignable<T>
{
  T temp = std::move(lhs);
  lhs = std::move(rhs);
  rhs = std::move(temp);
}
```

Note that the usage of the keyword `virtual` here is to indicate the fact that the function may be overridden elsewhere in code in a way that is dependent on the function's arguments. However, unlike traditional `virtual`, this is not achieved through dynamic dispatch. Whether or not this particualy syntax is a reasonable choice should be decided by committee if the overall functionality is considered useful.

§ 2.2. Overriding Customization Point Functions

A customization point function's primary use is as a non-member function that may be overridden. Unlike with normal function overloading, an override of a customization point function must be explicitly declared as an override of that specific customization point function. There are multiple ways in which a customization point function's override may be declared, but the simplest is to write the function type including the `override` specifier, followed by a `:` and the name of the customization point function that is to be overridden. The override declaration is also a definition if a function body immediately follows the name of the customization point function.

```
namespace std {

// A customization point function
template< class T >
virtual void swap( T& lhs, T& rhs ) { /*default definition*/ }

} // namespace std

// A user-defined type
class tensor { /*...*/ };

// A user-defined override for swap
void ( tensor& lhs, tensor& rhs ) override
  : std::swap // The customization point function to be overridden
{
  /*...*/
}
```

Informally, an override of a customization point function must be more specialized or more constrained than the customization point function that it overrides, but the return type of an override may differ if the override's return type is implicitly convertible to the customization point function's return type, or if the customization point's return type is `void`. Calls made to the customization point function force any such conversions to take place before returning to the caller.

Formal details surrounding these requirements and any alterations will be explored in a future revision if the rest of the proposal is positively received.

§ 2.2.1. Semantics of Customization Point Functions

When code attempts to call a customization point function, it is first resolved as if there were some imaginary function or function template with that declaration. If a call to such a function would be valid, then it and all of the direct overrides of that customization point function are treated as though they were all functions with the same name in some namespace with no other members. If a qualified call to a function of that name in that namespace would succeed, then the corresponding function is what is called in the actual program.

§ 2.3. Named Customization Point Function Overrides

An override of a customization point function may optionally be given a name. To specify a name for a customization point function override, the declaration is the same as a normal function declaration, except that it includes the override specifier and the customization point function that it overrides in the same manner as for an unnamed override described above. A name given to an overriding function definition must be a unique name in its namespace (it must be unique even with respect to other functions in that namespace).

```
template< class It, class Distance >
virtual constexpr void advance( It& it, Distance n )
  requires InputIterator<It>
{
  for( ; n != 0; --n )
    ++it;
}

template< class It, class Distance >
constexpr void advance_bidirectional( It& it, Distance n ) override
  requires BidirectionalIterator<It>
  : advance
{
  if( n >= 0 )
    for( ; n != 0; --n )
      ++it;
  else
    for( ; n != 0; ++n )
      --it;
}
```

One advantage of specification of a name for an override is that it makes it easier to invoke a specific override, much as it is useful to be able to invoke a specific `virtual` function by qualifying the call with the type name.

A named override is itself considered to also be a customization point function. Implications of this are described later on in this paper.

§ 2.4. Overrides as Friend Functions

In order to limit the amount of overrides that are looked up per call, it is suggested that a customization point function may be able to be overridden in the body of a class definition, in a similar manner to how an inline `friend` function definition behaves with respect to ADL. The details of this are to be explored in a revision of this paper if the overall idea is considered useful.

§ 2.5. Customization Point Functions as Objects

Despite the name, a customization point function is expected to behave more similar to a function object than to a function or function template. The rationale for this is to allow such a function to be able to be easily passed around to higher-order functions as a single entity. It also more accurately reflects intended behavior with respect to prohibition of "implicit" overloading and name lookup. A customization point function *cannot* be overloaded, but rather, it may only be overridden. Further, a customization point function is to not be found by ADL, and if when attempting to call a function by name a customization point function of that name is found by normal name lookup prior to ADL, then ADL is inhibited. The combination of these rules removes many of the subtleties of invoking customization points correctly.

§ 2.6. "Pure Virtual" Customization Point Functions

Similar to `virtual` functions, a customization point function may be declared pure virtual (although in contrast, a pure virtual customization point function may *not* have a definition). When attempting to call a pure virtual customization point function, if there is no override that matches the arguments that are passed, then overload resolution fails.

```
template< class T >
virtual void draw( context& cont, T const& obj ) = 0;
```

§ 2.7. Final Customization Point Functions

A customization point function may be declared `final`, in which case that function may not be overridden. Programs that attempt to declare an override for such a function are ill-formed. Though it may seem contradictory in intent to declare a customization point function as `final`, the primary use of declaring a customization point function as `final` is to take advantage of ADL-inhibition and the ability to safely and easily pass such functions to higher-order algorithms. This ability is also useful as

a means to counteract the understandable advocation by certain generic library authors to avoid namespace-scope functions entirely in favor of global function objects.[6]

A `final` customization point object may include the `virtual` specifier, but it is not required to do so.

```
template< class T >
T square( T const& arg ) final
{
  return arg * arg;
}

std::transform( range1, out_it, square );
```

§ 2.8. Hierarchical Customization Point Functions

As mentioned earlier, a named customization point function override is itself considered to be its own customization point function. This means that such an override may be explicitly overridden by name (it may also be pure and it may be `final`). This allows customization point functions to naturally form hierarchies of overrides, as opposed to a strictly flat set of overloads, in a way that may more clearly and more efficiently represent concept-based overloads.

```
template< class It, class Distance >
virtual constexpr void advance( It& it, Distance n )
  requires InputIterator<It>
{
  for( ; n != 0; --n )
    ++it;
}

template< class It, class Distance >
constexpr void advance_bidirectional( It& it, Distance n ) override
  requires BidirectionalIterator<It>
  : advance
{
  if( n >= 0 )
    for( ; n != 0; --n )
      ++it;
  else
    for( ; n != 0; ++n )
      --it;
}

template< class It, class Distance >
constexpr void advance_random_access( It& it, Distance n ) override
  requires RandomAccessIterator<It>
  : advance_bidirectional
{
  it += n;
}
```

Note that the above code example has some advantages over both traditional concept-based overloading and of branching via `if constexpr`. First, because the overrides are hierarchical, each level of override functions only needs to undergo substitution if the previous level succeeds substitution. In naturally hierarchical cases, this hypothetically may lead to better compile-time performance and/or simpler error messages when compared to normal concept-based overloads, though there is no implementation experience to verify this. As well, unlike with a nested `if constexpr` chain, the hierarchical overrides naturally form an open set of branches that is exposed to the user, and for which they may further customize.

A named customization point function override that is intended to be `final` must use the keyword `final` instead of `override` in its declaration.

§ 3. Suggested Polls

Should this proposal be elaborated on in a future revision?

Are the motivating problems worth solving at the language level at all?

Is the functionality of `final` non-member functions worth proposing separately?

§ 4. References

[1]: Matt Kulukundis: "Tip of the Week #152: AbslHashValue and You" https://abseil.io/tips/152

[2]: Eric Niebler: "Customization Point Design in C++11 and Beyond" http://ericniebler.com/2014/10/21/customization-point-design-in-c11-and-beyond/

[3]: Stack Overflow Search for ADL https://stackoverflow.com/search?q=ADL

[4]: N4762 "Working Draft, Standard for Programming Language C++" [customization.point.object]: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf

[5]: Jens Weller "An Interview with Sean Parent" https://www.meetingcpp.com/blog/items/interview-with-sean-parent.html

[6]: Eric Niebler: "C++11 Library Design" https://github.com/boostcon/cppnow_presentations_2014/blob/master/files/cxx11-library-design.pdf